

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО
ITMO University

Практическая работа 4

По дисциплине Проектирование и реализация баз данных

Тема работы Использование представлений, материализованных представлений и программируемых объектов базы данных в СУБД PostgreSQL..

Обучающийся Рекун Ирина Данииловна

Факультет прикладной информатики

Группа K3220

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся

(дата)

(подпись)

Рекун И.Д

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Войтюк Т.Е

(Ф.И.О.)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
Задание 1. Создание представления с помощью графического интерфейса.....	4
Задание 2. Создание представления на основе представления и изменение базовых таблиц представлений.....	5
Задание 3. Создание материализованного представления	7
Задание 4. Создание скалярной пользовательской функции	10
Задание 5. Создание собственной скалярной функции	11
Задание 6. Создание хранимой процедуры	12
Задание 7. Создание собственной хранимой процедуры.....	14
Выводы и анализ результатов работы	17

ВВЕДЕНИЕ

Цель работы: Изучение и закрепление навыков работы с представлениями, материализованными представлениями, пользовательскими функциями и хранимыми процедурами в СУБД PostgreSQL, а также освоение средств их создания и тестирования с использованием графического интерфейса pgAdmin 4 и языка SQL (DDL и DML).

Задачи, решаемые при выполнении работы:

1. Освоить создание обычных представлений (VIEW) с использованием графического интерфейса pgadmin 4.
2. Изучить создание представлений с помощью DDL-оператора CREATE VIEW.
3. Исследовать влияние представлений на возможность изменения базовых таблиц.
4. Создать и протестировать материализованное представление (MATERIALIZED VIEW).
5. Изучить различия между обычными и материализованными представлениями.
6. Создать и протестировать скалярную пользовательскую функцию.
7. Разработать собственную скалярную функцию с логикой ветвления.
8. Создать и протестировать хранимую процедуру.
9. Разработать собственную хранимую процедуру с несколькими режимами работы.

Задание 1. Создание представления с помощью графического интерфейса

Было создано представление `v_emp_active` в схеме `EmployeesDepartments` с использованием графического интерфейса `pgAdmin 4` (Рисунок 1.1). Данное представление отбирает только тех сотрудников, у которых заработная плата превышает 5000.

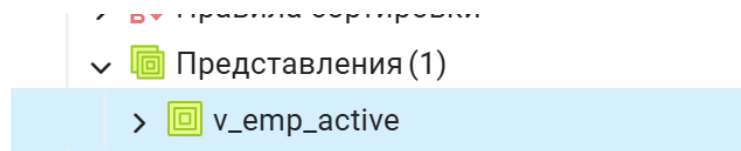


Рисунок 1.1 – представление `v_emp_active` в обозревателе объектов.

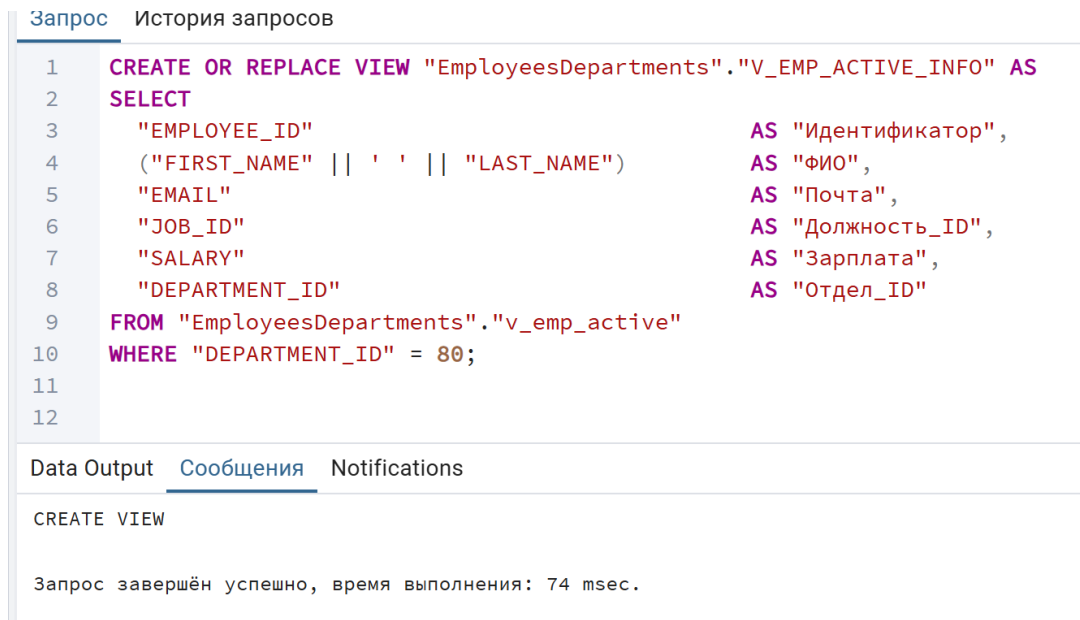
После создания представления `v_emp_active` была выполнена проверка его корректности путём выполнения запроса `SELECT` к данному представлению. В результате выполнения запроса был получен список сотрудников, заработная плата которых превышает 5000. Полученные данные соответствуют условиям, заданным при создании представления, что подтверждает его корректную работу. Результат выполнения запроса к представлению `v_emp_active` представлен на рисунке 1.2.

Data Output							
Showing rows: 1 to 24 of 1							
	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	JOB_ID	SALARY	DEPARTMENT_ID
	integer	character varying (20)	character varying (25)	character varying (25)	character varying (10)	numeric (8,2)	smallint
1	100	Steven	King	SKING	AD_PRES	24000.00	90
2	101	Neena	Kochhar	NKOCHHAR	AD_VP	17000.00	90
3	102	Lex	De Haan	LDEHAAN	AD_VP	17000.00	90
4	103	Alexander	Hunold	AHUNOLD	IT_PROG	9000.00	60
5	104	Bruce	Ernst	BERNST	IT_PROG	6000.00	60
6	124	Kevin	Mourgos	KMOURGOS	ST_MAN	5800.00	50
7	149	Eleni	Zlotkey	EZLOTKEY	SA_MAN	10500.00	80
8	174	Ellen	Abel	EABEL	SA_REP	11000.00	80
9	176	Jonathon	Taylor	JTAYLOR	SA_REP	8600.00	80
10	178	Kimberely	Grant	KGRANT	SA_REP	7000.00	[null]
11	201	Michael	Hartstein	MHARTSTE	MK_MAN	13000.00	20
12	205	Shelley	Higgins	SHIGGINS	AC_MGR	12000.00	110
13	206	William	Gietz	WGIEZT	AC_ACCOUNT	8300.00	110
14	207	Sophia	Barbosa Souza	SBARBOSASOUZA	SR_SA_REP	9500.00	85
15	208	Diego	Silva Pinto	DSILVAPINTO	SA_REP	7500.00	85
16	209	Sarah	Alves Rocha	SALVESROCHA	SA_REP	7300.00	85
17	210	Lucas	Almeida Castro	ALMEIDACASTRO	SA_REP	7300.00	85
18	212	Nick	Hooper	NHOOPER	SR_SA_REP	9600.00	80
19	215	Donna	Steiner	DSTEINER	SR_MK_REP	8600.00	20
20	222	Chen	Li	CLI	IT_PROG	8000.00	60
21	223	Alain	Fontaine	AFONTAINE	IT_PROG	7800.00	60
22	224	Matthias	Reinhard	MREINHARD	AC_ACCOUNT	8100.00	110
23	231	Jelena	Duric	JDURIC	AC_ACCOUNT	5400.00	110
24	232	Jennifer	Loermans	JLOERMANS	AC_ACCOUNT	5200.00	110

Рисунок 1.2 – Результат выполнения запроса к представлению `v_emp_active`

Задание 2. Создание представления на основе представления и изменение базовых таблиц представлений

Создаём представление V_EMP_ACTIVE_INFO на основе v_emp_active, для отдела 80 с заработной платой выше 5000. Определяем в окне “Запросника” DDL оператор CREATE для создания нового представления (Рисунок 2.1).



```
1 CREATE OR REPLACE VIEW "EmployeesDepartments"."V_EMP_ACTIVE_INFO" AS
2 SELECT
3     "EMPLOYEE_ID" AS "Идентификатор",
4     ("FIRST_NAME" || ' ' || "LAST_NAME") AS "ФИО",
5     "EMAIL" AS "Почта",
6     "JOB_ID" AS "Должность_ID",
7     "SALARY" AS "Зарплата",
8     "DEPARTMENT_ID" AS "Отдел_ID"
9 FROM "EmployeesDepartments"."v_emp_active"
10 WHERE "DEPARTMENT_ID" = 80;
11
12
```

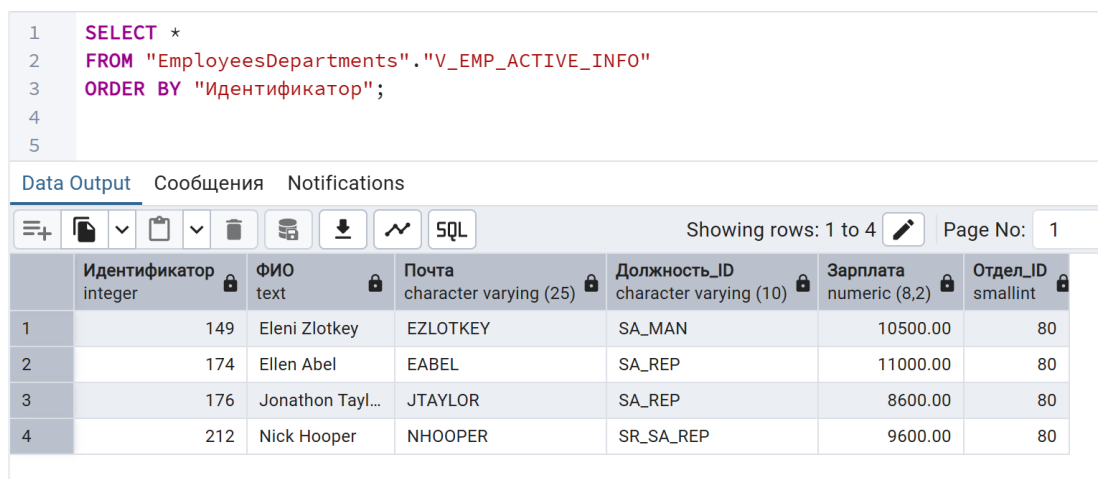
Data Output Сообщения Notifications

CREATE VIEW

Запрос завершён успешно, время выполнения: 74 msec.

Рисунок 2.1 – Создание представления V_EMP_ACTIVE_INFO

Далее выполняем запрос для того, чтобы протестировать представление V_EMP_ACTIVE_INFO. Результат выполнения запроса к представлению представлен на рисунке 2.2.



```
1 SELECT *
2 FROM "EmployeesDepartments"."V_EMP_ACTIVE_INFO"
3 ORDER BY "Идентификатор";
4
5
```

Data Output Сообщения Notifications

Showing rows: 1 to 4 Page No: 1

	Идентификатор integer	ФИО text	Почта character varying (25)	Должность_ID character varying (10)	Зарплата numeric (8,2)	Отдел_ID smallint
1	149	Eleni Zlotkey	EZLOTKEY	SA_MAN	10500.00	80
2	174	Ellen Abel	EABEL	SA_REP	11000.00	80
3	176	Jonathon Tayl...	JTAYLOR	SA_REP	8600.00	80
4	212	Nick Hooper	NHOOPER	SR_SA_REP	9600.00	80

Рисунок 2.2 – Результат выполнения запроса к представлению V_EMP_ACTIVE_INFO

Далее необходимо провести следующее тестирование "V_EMP_ACTIVE_INFO". Для этого добавляем одну запись непосредственно в таблицу "EMPLOYEEES", а вторую запись добавляем через

представление "v_emp_active". После этого был выполняем запрос к представлению V_EMP_ACTIVE_INFO (Рисунок 2.3).

```

1  INSERT INTO "EmployeesDepartments"."EMPLOYEES"
2  ("FIRST_NAME", "LAST_NAME", "EMAIL", "JOB_ID", "SALARY", "DEPARTMENT_ID")
3  VALUES
4  ('Ivan', 'Ivanov', 'Iv_Ivn@itmo.ru', 'SA_REP', 6000, 80);
5
6  INSERT INTO "EmployeesDepartments"."v_emp_active"
7  ("FIRST_NAME", "LAST_NAME", "EMAIL", "JOB_ID", "SALARY", "DEPARTMENT_ID")
8  VALUES
9  ('Petr', 'Petrov', 'Pe_Ptr@itmo.ru', 'AD_VP', 22000, 80);
10
11 SELECT *
12 FROM "EmployeesDepartments"."V_EMP_ACTIVE_INFO"
13 ORDER BY "Идентификатор";

```

	Идентификатор integer	ФИО text	Почта character varying (25)	Должность_ID character varying (10)	Зарплата numeric (8,2)	Отдел_ID smallint
1	149	Eleni Zlotkey	EZLOTKEY	SA_MAN	10500.00	80
2	174	Ellen Abel	EABEL	SA_REP	11000.00	80
3	176	Jonathon Tayl...	JTAYLOR	SA_REP	8600.00	80
4	212	Nick Hooper	NHOOPER	SR_SA_REP	9600.00	80
5	236	Ivan Ivanov	Iv_Ivn@itmo.ru	SA_REP	6000.00	80
6	237	Petr Petrov	Pe_Ptr@itmo.ru	AD_VP	22000.00	80

Рисунок 2.3 – Проверка представления V_EMP_ACTIVE_INFO после добавления данных в таблицу.

В результате было установлено, что обычное представление не хранит данные физически, а все операции INSERT, выполненные через представление, приводят к добавлению данных в базовую таблицу EMPLOYEES.

После попытаемся внести изменения в таблицу "EMPLOYEES" с помощью кода и проанализируем ошибку при изменении базовой таблицы (Рисунок 2.4).

```

1  ALTER TABLE "EmployeesDepartments"."EMPLOYEES"
2  ALTER COLUMN "SALARY" TYPE NUMERIC(10,2);
3

```

Data Output	Сообщения	Notifications
ERROR: изменить тип столбца, задействованного в представлении или правиле, нельзя правило _RETURN для отношения представление v_emp_active зависит от столбца "SALARY"		
ОШИБКА: изменить тип столбца, задействованного в представлении или правиле, нельзя SQL-состояние: 0A000		
Подробности: правило _RETURN для отношения представление v_emp_active зависит от столбца "SALARY"		

Таким образом, при попытке изменить структуру таблицы EMPLOYEES, столбцы которой используются в представлении, будет получено сообщение об ошибке зависимости. Это демонстрирует, что представления зависят от структуры базовых таблиц.

Задание 3. Создание материализованного представления

Пересоздаем существующее представление с использованием директивы MATERIALIZED (Рисунок 3.1). При попытке удаления представления v_emp_active была получена ошибка, связанная с наличием зависимого объекта V_EMP_ACTIVE_INFO. Это обусловлено тем, что представление второго уровня использует данные представления v_emp_active. Для корректного удаления объектов сначала было удалено зависимое представление, после чего основное представление было удалено без ошибок.

```
1 DROP VIEW "EmployeesDepartments"."V_EMP_ACTIVE_INFO";
2 DROP VIEW "EmployeesDepartments"."v_emp_active";
3
4
5 CREATE MATERIALIZED VIEW "EmployeesDepartments"."v_emp_active" AS
6 SELECT
7     "EMPLOYEE_ID",
8     "FIRST_NAME",
9     "LAST_NAME",
10    "EMAIL",
11    "JOB_ID",
12    "SALARY",
13    "DEPARTMENT_ID"
14 FROM "EmployeesDepartments"."EMPLOYEES"
15 WHERE "SALARY" > 5000;
16
```

Data Output [Сообщения](#) Notifications

SELECT 26

Рисунок 3.1 – Создание материализованного представления

После создания материализованного представления было выполнено его тестирование. Для этого был выполнен запрос, выбирающий данные сотрудника с фамилией Ivanov (Рисунок 3.2).

```

1  SELECT
2      "EMPLOYEE_ID",
3      "FIRST_NAME",
4      "LAST_NAME",
5      "EMAIL",
6      "SALARY"
7  FROM "EmployeesDepartments"."v_emp_active"
8  WHERE "LAST_NAME" = 'Ivanov';
9
10

```

Data Output Сообщения Notifications

Showing rows: 1 to 1 Page No: 1

	EMPLOYEE_ID integer	FIRST_NAME character varying (20)	LAST_NAME character varying (25)	EMAIL character varying (25)	SALARY numeric (8,2)
1	236	Ivan	Ivanov	lv_lvn@itmo.ru	6000.00

Рисунок 3.2 – Результат выборки данных сотрудника с фамилией Ivanov из материализованного представления.

Далее было изменено имя сотрудника с фамилией Ivanov в таблице EMPLOYEES, установив ему имя Petr и выполнена проверка что данные в таблице EMPLOYEES действительно изменились (Рисунок 3.3).

```

1  SELECT
2      "EMPLOYEE_ID",
3      "FIRST_NAME",
4      "LAST_NAME",
5      "EMAIL",
6      "SALARY"
7  FROM "EmployeesDepartments"."EMPLOYEES"
8  WHERE "LAST_NAME" = 'Ivanov';
9

```

Data Output Сообщения Notifications

Showing rows: 1 to 1 Page 1

	EMPLOYEE_ID [PK] integer	FIRST_NAME character varying (20)	LAST_NAME character varying (25)	EMAIL character varying (25)	SALARY numeric (8,2)
1	236	Petr	Ivanov	lv_lvn@itmo.ru	6000.00

Рисунок 3.3 – Проверка изменения значения в таблице "EMPLOYEES"

После выполняем такую же проверку, но уже из MATERIALIZED VIEW (Рисунок 3.4).

1
2
3
4
5
6
7
8
9
10

SELECT
"EMPLOYEE_ID",
"FIRST_NAME",
"LAST_NAME",
"EMAIL",
"SALARY"
FROM "EmployeesDepartments"."v_emp_active"
WHERE "LAST_NAME" = 'Ivanov';

Data Output Сообщения Notifications

+

SQL

Showing rows: 1 to 1

Page No

EMPLOYEE_ID integer	FIRST_NAME character varying (20)	LAST_NAME character varying (25)	EMAIL character varying (25)	SALARY numeric (8,2)	
1	236	Ivan	Ivanov	Iv_lvn@itmo.ru	6000.00

Рисунок 3.4 – Проверка изменения значения в материализованном представлении v_emp_active

Таким образом, материализованное представление не обновляется автоматически при изменении данных в исходных таблицах.

Чтобы получить актуальную информацию из материализованного представления, обновляем его вручную с помощью команды REFRESH MATERIALIZED VIEW. После выполнения обновления данные в материализованном представлении стали соответствовать таблице EMPLOYEES. (Рисунок 3.5).

1

REFRESH MATERIALIZED VIEW "EmployeesDepartments"."v_emp_active";

2

3

SELECT

4

"EMPLOYEE_ID",

5

"FIRST_NAME",

6

"LAST_NAME",

7

"EMAIL",

8

"SALARY"

9

FROM "EmployeesDepartments"."v_emp_active"

10

WHERE "LAST_NAME" = 'Ivanov';

11

Data Output

Сообщения

Notifications

≡

📄

▼

📋

▼

🗑

🗄

⬇

📈

SQL

Showing rows: 1 to 1

✎

Page No:

	EMPLOYEE_ID integer	FIRST_NAME character varying (20)	LAST_NAME character varying (25)	EMAIL character varying (25)	SALARY numeric (8,2)
1	236	Petr	Ivanov	lv_lvn@itmo.ru	6000.00

Рисунок 3.5 – Обновление материализованного представления. Результат после проверки.

В ходе выполнения задания было установлено, что обычное представление (VIEW) не хранит данные физически и при каждом обращении формирует результат на основе актуальных данных таблиц, тогда как материализованное представление (MATERIALIZED VIEW) сохраняет результат запроса в базе данных и не обновляется автоматически при изменении исходных данных. Для обновления информации в

материализованном представлении требуется выполнение команды REFRESH MATERIALIZED VIEW. Обычные представления лучше использовать при необходимости получения всегда актуальных данных, а материализованные — при работе с объёмными или ресурсоёмкими запросами, когда важна скорость выполнения.

Задание 4. Создание скалярной пользовательской функции

Была создана пользовательская функция, возвращающая полное имя сотрудника по его идентификатору (Рисунок 4.1). Функция принимает идентификатор сотрудника в качестве параметра и возвращает строку, содержащую имя и фамилию сотрудника. Если сотрудник не найден, то функция должна вернуть значение NULL.

```
1 CREATE OR REPLACE FUNCTION
2 "EmployeesDepartments".get_employee_full_name(p_emp_id INT)
3 RETURNS TEXT AS
4 $$
5 DECLARE
6     v_full_name TEXT;
7 BEGIN
8     SELECT "FIRST_NAME" || ' ' || "LAST_NAME"
9     INTO v_full_name
10    FROM "EmployeesDepartments"."EMPLOYEES"
11    WHERE "EMPLOYEE_ID" = p_emp_id;
12
13    RETURN v_full_name;
14 END;
15 $$ LANGUAGE plpgsql;
```

Data Output Сообщения Notifications

CREATE FUNCTION

Рисунок 4.1 – Создание скалярной пользовательской функции.

После необходимо протестировать работоспособность скалярной функции, передав в нее известный идентификатор сотрудника (Рисунок 4.2).

Запрос История запросов

```
1 SELECT
2 "EmployeesDepartments".get_employee_full_name(236);
3
4
```

Data Output Сообщения Notifications

	get_employee_full_name text
1	Petr Ivanov

Showing 1

Рисунок 4.2 – Результат вызова функции get_employee_full_name

В результате выполнения запроса было получено полное имя сотрудника, что подтверждает корректную работу пользовательской функции.

Задание 5. Создание собственной скалярной функции

Необходимо создать собственную скалярную функцию и описать её назначение. Требования к функции:

1. Функция должна принимать два параметра. Первый параметр используется для отбора строк из выбранной таблицы (например, по EMPLOYEE_ID, DEPARTMENT_ID или другому признаку). Второй параметр задаёт режим работы функции (например: выполнять округление заработной платы или возвращать точное значение; учитывать комиссию или нет; приводить текст к верхнему/нижнему регистру и т.д.).
2. Внутри функции должен быть реализован алгоритм с использованием оператора ветвления «IF...ELSE...END IF» или «CASE», если логику ветвления можно выразить в SQL-запросе.
3. Если подходящие строки не найдены, функция должна возвращать.

Таким образом, мною была разработана собственная скалярная функция, возвращающая заработную плату сотрудника в зависимости от переданного режима работы. Код для создания этой функции представлен на рисунке 5.1.

```
1 CREATE OR REPLACE FUNCTION "EmployeesDepartments".get_salary_mode(  
2     p_emp_id INT,  
3     p_mode INT  
4 )  
5 RETURNS NUMERIC  
6 LANGUAGE plpgsql  
7 AS $$  
8 DECLARE  
9     v_salary NUMERIC;  
10 BEGIN  
11     SELECT "SALARY"  
12     INTO v_salary  
13     FROM "EmployeesDepartments"."EMPLOYEES"  
14     WHERE "EMPLOYEE_ID" = p_emp_id;  
15  
16     IF p_mode = 1 THEN  
17         RETURN ROUND(v_salary);  
18     ELSE  
19         RETURN v_salary;  
20     END IF;  
21 END;  
22 $$;  
23  
24  
25
```

Data Output [Сообщения](#) Notifications

CREATE FUNCTION

Рисунок 5.1 – Создание собственной скалярной функции get_salary_mode

Функция была протестирована в двух разных режимах. Полученный результат тестирования показан на рисунке 5.2.

The screenshot shows a SQL query editor with the following code:

```
1 SELECT
2 "EmployeesDepartments".get_salary_mode(236, 0) AS "Зарплата",
3 "EmployeesDepartments".get_salary_mode(236, 1) AS "Округленная зарплата";
4
5
6
7
```

Below the query editor, the 'Data Output' tab is active, displaying the results in a table. The table has two columns: 'Зарплата' (Salary) and 'Округленная зарплата' (Rounded Salary). The first row shows the results for the query.

	Зарплата numeric	Округленная зарплата numeric
1	6000.00	6000

Рисунок 5.2 – Результаты тестирования функции get_salary_mode

Полученные значения объясняются логикой работы функции get_salary_mode. При значении параметра режима 0 функция возвращает фактическое значение заработной платы сотрудника без изменений. При значении режима 1 используется округление заработной платы, в результате чего значение приводится к целому числу. В рассматриваемом случае исходная заработная плата сотрудника составляет 6000, что при округлении не изменяет значение, поэтому в обоих режимах получено одинаковое число.

Задание 6. Создание хранимой процедуры

В данном задании создаётся хранимая процедура, предназначенная для увеличения заработной платы выбранного сотрудника на указанный процент. Процедура отличается от функции тем, что предназначена для выполнения действий (обновления данных) и не возвращает значение, поэтому в её объявлении отсутствует ключевое слово RETURN. Логика процедуры заключается в выполнении оператора UPDATE, который изменяет значение поля "SALARY" в таблице "EMPLOYEES". Код создания хранимой процедуры представлен на рисунке 6.1.

1	CALL "EmployeesDepartments".raise_salary(236, 10);
2	SELECT "EMPLOYEE_ID", "LAST_NAME", "SALARY"
3	FROM "EmployeesDepartments"."EMPLOYEES"
4	WHERE "EMPLOYEE_ID" = 236;
5	

Data Output	Сообщения	Notifications
-------------	-----------	---------------

EMPLOYEE_ID	LAST_NAME	SALARY
[PK] integer	character varying (25)	numeric (8,2)
1	236	Ivanov
		6600.00

Рисунок 6.3 – Заработная плата сотрудника после вызова процедуры.

В PostgreSQL параметры процедуры можно передавать не только по порядку, но и по имени. Это позволяет менять порядок параметров при вызове. Таким образом, вызвав хранимую процедуру salary с передачей параметров по имени проверяем изменилась ли заработная плата того же сотрудника.

Запрос	История запросов
1	CALL "EmployeesDepartments".raise_salary(p_percent:=10, p_employee_id:=236);
2	SELECT "EMPLOYEE_ID", "LAST_NAME", "SALARY"
3	FROM "EmployeesDepartments"."EMPLOYEES"
4	WHERE "EMPLOYEE_ID" = 236;
5	

Data Output	Сообщения	Notifications
-------------	-----------	---------------

EMPLOYEE_ID	LAST_NAME	SALARY
[PK] integer	character varying (25)	numeric (8,2)
1	236	Ivanov
		7986.00

Рисунок 6.3 – Вызов процедуры raise_salary с передачей параметров по имени. Изменение успешно.

Задание 7. Создание собственной хранимой процедуры

В данном задании была разработана собственная хранимая процедура, выполняющая одно из двух действий в зависимости от режима работы. Процедура принимает два параметра: идентификатор сотрудника и режим. Внутри процедуры реализовано ветвление IF...ELSE...END IF, позволяющее выполнять разные SQL-операции над таблицами. Если подходящая строка не найдена, процедура корректно завершает работу (обновление затронет 0 строк).

Реализованы следующие режимы работы процедуры:

- ❖ **режим 1** — назначение сотруднику комиссионного процента (COMMISSION_PCT = 0.1);
- ❖ **режим 2** — удаление комиссионного процента (COMMISSION_PCT = NULL).

Код реализации собственной хранимой процедуры отображен на рисунке 7.1.

```
1 CREATE OR REPLACE PROCEDURE "EmployeesDepartments".manage_employee_commission(  
2     p_employee_id INTEGER,  
3     p_mode INTEGER  
4 )  
5 LANGUAGE plpgsql  
6 AS $$  
7 BEGIN  
8     IF p_mode = 1 THEN  
9         -- Режим 1: назначить комиссионный процент  
10        UPDATE "EmployeesDepartments"."EMPLOYEES"  
11        SET "COMMISSION_PCT" = 0.1  
12        WHERE "EMPLOYEE_ID" = p_employee_id;  
13  
14    ELSE  
15        -- Режим 2: убрать комиссионный процент  
16        UPDATE "EmployeesDepartments"."EMPLOYEES"  
17        SET "COMMISSION_PCT" = NULL  
18        WHERE "EMPLOYEE_ID" = p_employee_id;  
19    END IF;  
20 END;  
21 $$;  
22
```

Data Output Сообщения Notifications

CREATE PROCEDURE

Рисунок 7.1 – Создание собственной хранимой процедуры manage_employee_commission

В зависимости от значения параметра p_mode процедура выполняет различные SQL-операции над таблицей "EMPLOYEES". При значении режима 1 сотруднику назначается комиссионный процент. При значении режима 2 комиссионный процент удаляется. Если сотрудник с указанным идентификатором отсутствует, обновление не выполняется, и процедура завершается корректно.

Для того, чтобы проверить работу собственной хранимой процедуры, зафиксируем данные до вызова процедуры (Рисунок 7.2)

Запрос История запросов	
1	SELECT "EMPLOYEE_ID", "LAST_NAME", "COMMISSION_PCT"
2	FROM "EmployeesDepartments"."EMPLOYEES"
3	WHERE "EMPLOYEE_ID" = 236;
4	
5	
Data Output Сообщения Notifications	
Showing row	
	EMPLOYEE_ID [PK] integer
	LAST_NAME character varying (25)
	COMMISSION_PCT numeric (2,2)
1	236 Ivanov [null]

Рисунок 7.2 – Данные сотрудника до выполнения процедуры

Вызываем хранимую процедуру в режиме 1 (назначение комиссионного процента) и видим успешное выполнение процедуры и изменение данных (Рисунок 7.3)

Запрос История запросов	
1	CALL "EmployeesDepartments".manage_employee_commission(236, 1);
2	SELECT "EMPLOYEE_ID", "LAST_NAME", "COMMISSION_PCT"
3	FROM "EmployeesDepartments"."EMPLOYEES"
4	WHERE "EMPLOYEE_ID" = 236;
5	
Data Output Сообщения Notifications	
Showing rows: 1 to	
	EMPLOYEE_ID [PK] integer
	LAST_NAME character varying (25)
	COMMISSION_PCT numeric (2,2)
1	236 Ivanov 0.10

Рисунок 7.3 – Результат работы процедуры в режиме 1

Далее протестируем процедуру в режиме 2 (удаление комиссионного процента). Результат тестирования показан на рисунке 7.4.

Запрос История запросов	
1	CALL "EmployeesDepartments".manage_employee_commission(236, 2);
2	
3	SELECT "EMPLOYEE_ID", "LAST_NAME", "COMMISSION_PCT"
4	FROM "EmployeesDepartments"."EMPLOYEES"
5	WHERE "EMPLOYEE_ID" = 236;
6	
Data Output Сообщения Notifications	
Showing rows: 1 to	
	EMPLOYEE_ID [PK] integer
	LAST_NAME character varying (25)
	COMMISSION_PCT numeric (2,2)
1	236 Ivanov [null]

Рисунок 7.4 – Результат работы процедуры в режиме 2.

Выводы и анализ результатов работы

В ходе выполнения практической работы были выполнены все поставленные в методических указаниях задания, направленные на изучение и практическое освоение механизмов представлений, материализованных представлений, пользовательских функций и хранимых процедур в СУБД PostgreSQL.

В процессе работы были созданы и протестированы обычные представления и материализованные представления, что позволило на практике изучить различия между ними. Было установлено, что обычные представления всегда отображают актуальные данные исходных таблиц, тогда как материализованные представления сохраняют результат запроса на момент создания или обновления и не изменяются автоматически при изменении данных в базовых таблицах.

Также в рамках работы была разработана собственная скалярная функция с несколькими режимами работы. Функция была протестирована в разных режимах, что позволило убедиться в корректности реализованной логики и правильности возвращаемых значений.

Отдельное внимание было уделено созданию и использованию хранимых процедур. В ходе работы была создана процедура для изменения данных сотрудников, а также собственная хранимая процедура с несколькими режимами работы, выполняющая различные SQL-операции в зависимости от переданных параметров. Корректность работы процедур была подтверждена вызовами с различными параметрами и анализом изменений данных до и после их выполнения.

В процессе выполнения практической работы возникли некоторые трудности. Основные сложности были связаны с зависимостями между объектами базы данных (например, невозможность удаления представления без учёта зависимых объектов), а также с особенностями работы материализованных представлений, которые не обновляются автоматически. Указанные проблемы были успешно решены путём анализа сообщений об ошибках PostgreSQL и применения соответствующих операторов SQL.

Таким образом, цель практической работы была полностью достигнута. В результате выполнения заданий были получены практические навыки создания и использования представлений, материализованных представлений, пользовательских функций и хранимых процедур в PostgreSQL.